

INFO284 Machine Learning Group Exam - Spring 2026

Task 1: Sentiment Analysis

In this notebook we build a pipeline for predicting review scores from review text. After exploring and cleaning the data, we train and evaluate four models:

1. **Logistic Regression**
2. **Support Vector Machine (LinearSVC)**
3. **LightGBM**
4. **Bidirectional LSTM**

1. Imports and Setup

```
In [1]: import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import re
import warnings
import random
import nlpaug.augmenter.word as naw
import lightgbm as lgb

from collections import Counter
from pathlib import Path

from sklearn.model_selection import train_test_split, cross_val_score, GridSearchCV
from sklearn.feature_extraction.text import TfidfVectorizer, CountVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
from sklearn.pipeline import Pipeline
from sklearn.metrics import (
    classification_report, confusion_matrix, ConfusionMatrixDisplay,
    accuracy_score, precision_recall_curve, average_precision_score
)
from sklearn.utils.class_weight import compute_class_weight
from sklearn.preprocessing import label_binarize

import nltk
from nltk.corpus import stopwords
nltk.download('stopwords', quiet=True)
nltk.download('wordnet', quiet=True)
nltk.download('averaged_perceptron_tagger_eng', quiet=True)

import tensorflow as tf
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
```

```

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Bidirectional, Dense
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.callbacks import EarlyStopping
from tensorflow.keras import layers, models
from tensorflow.keras.applications import EfficientNetB0
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.models import load_model
from tensorflow.keras.utils import load_img, img_to_array

warnings.filterwarnings('ignore')
tf.random.set_seed(42)
np.random.seed(42)
random.seed(42)

dataset_path = 'Art_shuffled/'

```

WARNING: All log messages before absl::InitializeLog() is called are written to STDERR

I0000 00:00:1777472791.439731 32897 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF\_ENABLE\_ONEDNN\_OPTS=0`.

I0000 00:00:1777472791.472963 32897 cpu_feature_guard.cc:227] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.

To enable the following instructions: AVX2 AVX_VNNI FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.

WARNING: All log messages before absl::InitializeLog() is called are written to STDERR

I0000 00:00:1777472792.536794 32897 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF\_ENABLE\_ONEDNN\_OPTS=0`.

I0000 00:00:1777472792.537181 32897 cudart_stub.cc:31] Could not find cuda drivers on your machine, GPU will not be used.

2. Data Loading

```

In [2]: df = pd.read_csv("reviews.csv")
df['review_len'] = df['review_text'].str.len()

print(f"Total reviews: {len(df)}")
print(f"Columns: {list(df.columns)}")
df.head(10)

```

Total reviews: 6210
Columns: ['review_id', 'rating', 'review_text', 'review_date', 'helpful', 'review_len']

Out[2]:

	review_id	rating	review_text	review_date	helpful
0	981e465b-d3ba-4632-9c60-25051efac38a	5	It's good	11/22/2025 1:19	0
1	964d3555-9429-4c20-8127-ce3c71ce9273	5	WhatsApp not working well always shows offline...	11/24/2025 20:03	0
2	6c28859f-1554-4ca1-9aa8-9d66f204be0a	5	Oppo not corresponding, share with me the offi...	11/25/2025 6:26	0
3	a7efafc3-5871-4020-a398-9cc12cb4072a	5	Excellent app, great communication super conne...	11/25/2025 18:09	0
4	de142b31-a5ad-446f-b7c8-51b264728478	4	simply the Best for chat and calls.i love it	11/24/2025 1:10	1
5	8b0ffc3b-d7d0-4198-8ce0-763674359246	5	good. but i need WhatsApp premium features	11/26/2025 16:42	0
6	XCdQN9L5-PsBJ-ftUV-Ov6H-Na1v3f0JxTLs	4	learning learning learning learning l...	25/11/2025 06:54	1
7	18294fa1-0410-4476-970f-33f188b0abfe	5	Awesome. I just need it to download and install	11/22/2025 20:27	0
8	c9fd6209-d8ed-482e-9b03-68cc2100fed0	5	very nice app thnx so much	11/22/2025 15:53	0
9	75b5068d-6b9b-4aa6-a069-abdb6bfc6388	5	Really really apriacite 100/	11/26/2025 17:06	0

In [3]: `df.info()`

```
<class 'pandas.DataFrame'>
RangeIndex: 6210 entries, 0 to 6209
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype
---  -
0   review_id       6210 non-null   str
1   rating          6210 non-null   int64
2   review_text     6210 non-null   str
3   review_date     6210 non-null   str
4   helpful         6210 non-null   int64
5   review_len      6210 non-null   int64
dtypes: int64(3), str(3)
memory usage: 291.2 KB
```

3. Exploratory Data Analysis

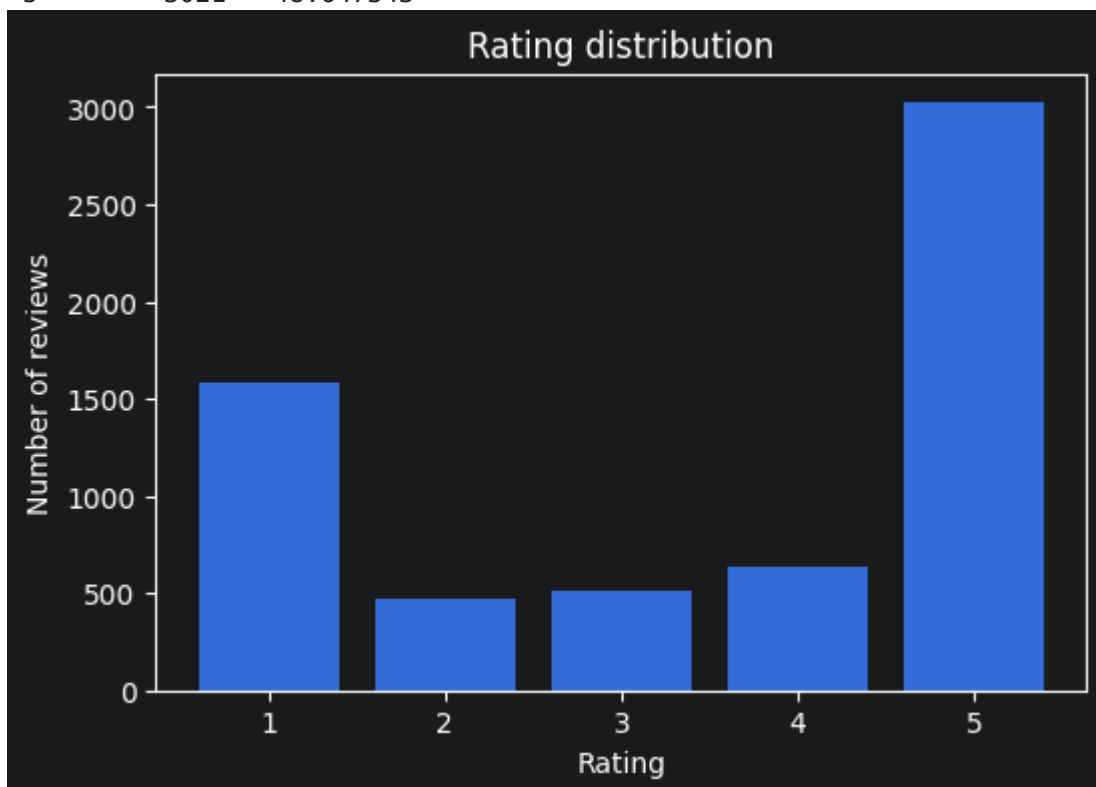
We start by understanding the rating distribution, review lengths, and word frequencies. This informs our preprocessing choices and helps anticipate challenges like class imbalance.

3.1 Rating distribution

```
In [4]: rating_stats = pd.DataFrame({
        'Count': df['rating'].value_counts().sort_index(),
        'Percentage': df['rating'].value_counts(normalize=True).sort_index()
    })
print(rating_stats)

plt.figure(figsize=(6, 4))
plt.bar(rating_stats.index, rating_stats['Count'])
plt.xlabel("Rating")
plt.ylabel("Number of reviews")
plt.title("Rating distribution")
plt.show()
```

	Count	Percentage
rating		
1	1582	25.475040
2	467	7.520129
3	508	8.180354
4	632	10.177134
5	3021	48.647343



The dataset is heavily imbalanced: rating 5 makes up almost half of all reviews, while ratings 2 and 3 have very few samples. A model that always predicts 5 would already achieve nearly 50% accuracy without learning anything meaningful. We address this with class weighting, text augmentation (LSTM only), and by using macro-averaged F1 as the primary metric.

3.2 Review Length

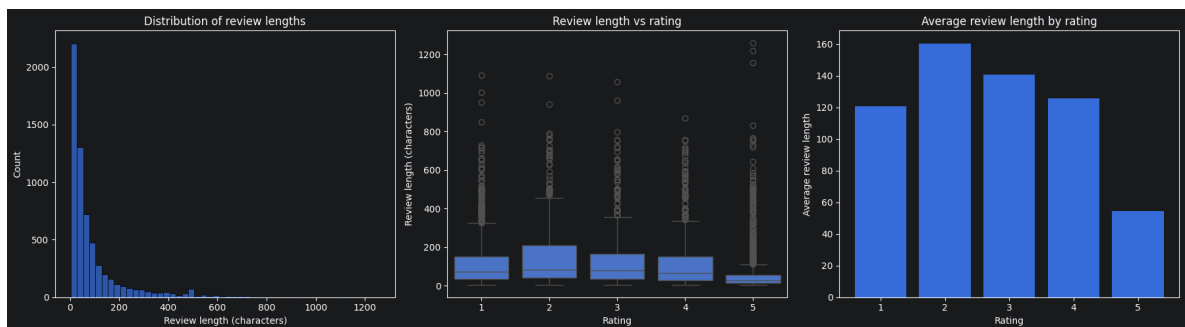
```
In [5]: fig, axes = plt.subplots(1, 3, figsize=(18, 5))

sns.histplot(df['review_len'], bins=50, ax=axes[0])
axes[0].set_xlabel("Review length (characters)")
axes[0].set_title("Distribution of review lengths")

sns.boxplot(x='rating', y='review_len', data=df, ax=axes[1])
axes[1].set_xlabel("Rating")
axes[1].set_ylabel("Review length (characters)")
axes[1].set_title("Review length vs rating")

avg_length = df.groupby('rating')['review_len'].mean()
axes[2].bar(avg_length.index, avg_length.values)
axes[2].set_xlabel("Rating")
axes[2].set_ylabel("Average review length")
axes[2].set_title("Average review length by rating")

plt.tight_layout()
plt.show()
```



Review length is right-skewed: most reviews are short, with a long tail of detailed ones. 1-star reviews tend to be longer (more detailed complaints), while 5-star reviews cluster at shorter lengths. Length carries some signal, but we leave it out as a feature and let the models learn from text directly. This analysis also informs the LSTM's `MAX_LEN=100` : most reviews fall under this token count after stopword removal, so truncation only affects a small fraction of samples.

3.3 Word and Bigram Frequencies

```
In [6]: _eda_stop = set(stopwords.words('english')) - {"not", "no", "nor", "never"}

def _eda_tokenize(text):
    text = text.lower()
    text = re.sub(r"^\w\s", "", text)
    return [w for w in text.split() if w not in _eda_stop]

positive_reviews = df[df['rating'] >= 4]
negative_reviews = df[df['rating'] <= 2]

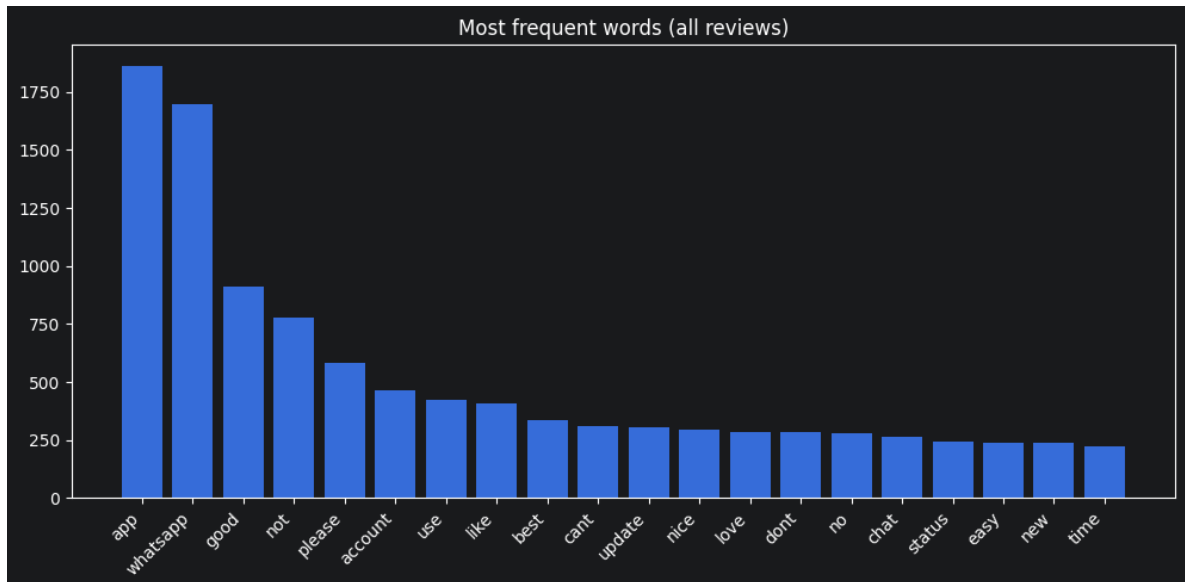
all_tokens = []
for review in df['review_text']:
    all_tokens.extend(_eda_tokenize(review))
```

```

common_words = Counter(all_tokens).most_common(20)
words, counts = zip(*common_words)

plt.figure(figsize=(10, 5))
plt.bar(words, counts)
plt.xticks(rotation=45, ha='right')
plt.title("Most frequent words (all reviews)")
plt.tight_layout()
plt.show()

```



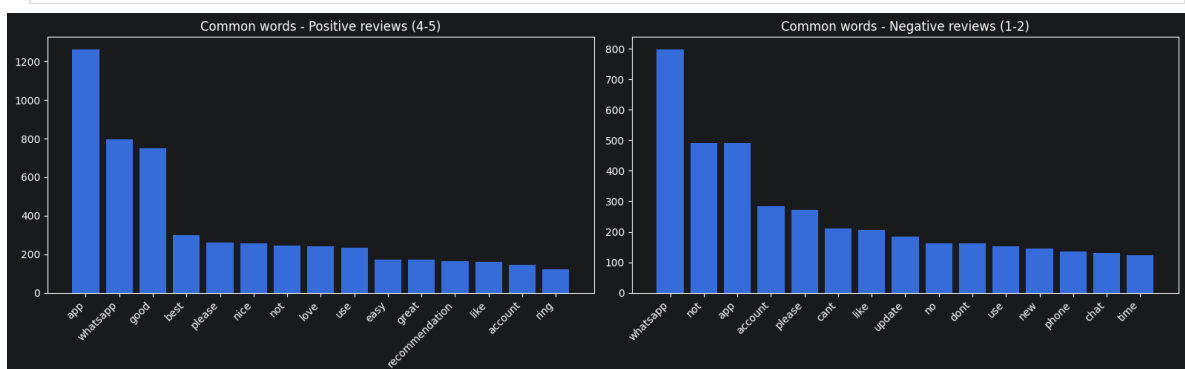
```

In [7]: pos_tokens, neg_tokens = [], []
for review in positive_reviews['review_text']:
    pos_tokens.extend(_eda_tokenize(review))
for review in negative_reviews['review_text']:
    neg_tokens.extend(_eda_tokenize(review))

fig, axes = plt.subplots(1, 2, figsize=(16, 5))
for ax, tokens, title in zip(axes,
    [pos_tokens, neg_tokens],
    ["Positive reviews (4-5)", "Negative reviews (1-2)"]):
    words, counts = zip(*Counter(tokens).most_common(15))
    ax.bar(words, counts)
    ax.set_xticklabels(words, rotation=45, ha='right')
    ax.set_title(f"Common words - {title}")

plt.tight_layout()
plt.show()

```



```

In [8]: vectorizer_bi = CountVectorizer(stop_words='english', ngram_range=(2, 2),
X_bi = vectorizer_bi.fit_transform(df['review_text'])

```

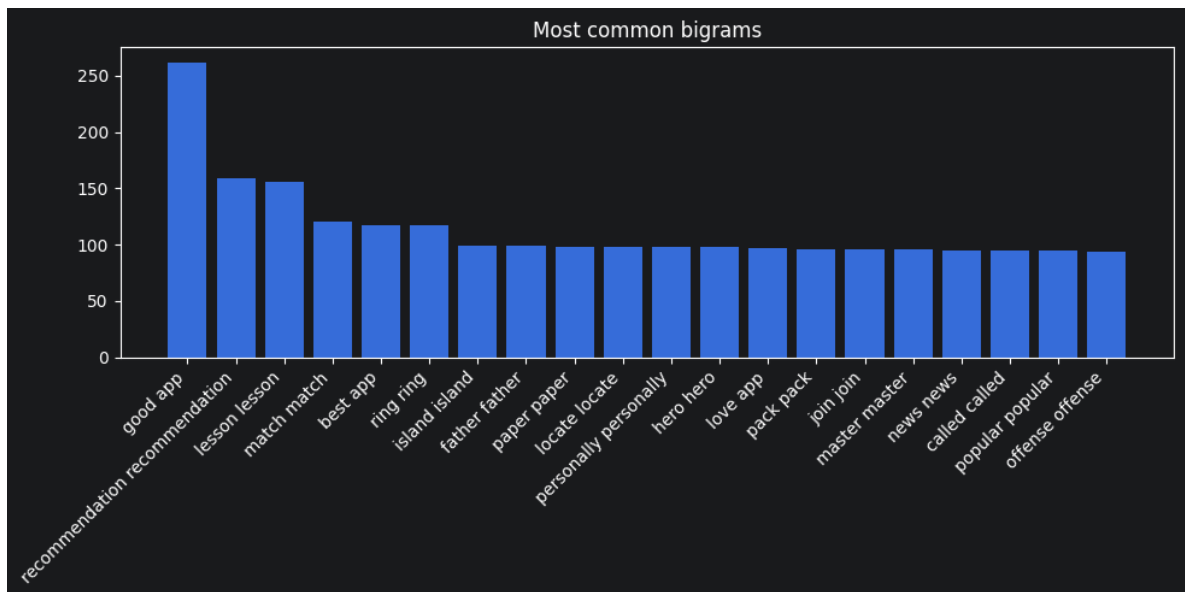
```

sums = X_bi.sum(axis=0)

bigrams = sorted(
    [(word, sums[0, idx]) for word, idx in vectorizer_bi.vocabulary_.items()
     key=lambda x: x[1], reverse=True]
)
words, counts = zip(*bigrams)

plt.figure(figsize=(10, 5))
plt.bar(words, counts)
plt.xticks(rotation=45, ha='right')
plt.title("Most common bigrams")
plt.tight_layout()
plt.show()

```



The most frequent unigrams include sentiment-laden words, but also domain noise like "app" and "whatsapp", which appear often without carrying sentiment, we therefore add these to the stopwords list. The same analysis motivates keeping negation words, since they appear frequently and flip surrounding sentiment. The bigram analysis reveals nonsensical repeating sequences in many reviews, which we address in the cleaning pipeline below.

4. Data Quality Checks and Cleaning

```

In [9]: def is_gibberish(text):
        if not isinstance(text, str) or len(text.strip()) == 0:
            return True
        words = text.split()
        long_token = any(len(w) > 30 for w in words)
        alnum_ratio = sum(c.isalpha() for c in text) / max(len(text), 1)
        return long_token or alnum_ratio < 0.5

def is_spam_repetition(text):
    if not isinstance(text, str):
        return False
    words = text.lower().split()
    if len(words) < 3:
        return False

```

```

most_common_count = Counter(words).most_common(1)[0][1]
return (most_common_count / len(words)) > 0.5

def is_non_english(text):
    if not isinstance(text, str) or len(text.strip()) < 8:
        return False
    stripped = re.sub(r'^\u0000-\u024F', '', text).strip()
    original_no_emoji = re.sub(r'^\u0000-\u024F\s', '', text).strip()
    ratio = len(stripped) / max(len(original_no_emoji), 1)
    return ratio < 0.7

def is_emoji_spam(text):
    if not isinstance(text, str):
        return False
    emoji_count = len(re.findall(r'^\x00-\u024F', text))
    total_chars = max(len(text.strip()), 1)
    return (emoji_count / total_chars) > 0.8

df['is_gibberish'] = df['review_text'].apply(is_gibberish)
df['is_spam_repetition'] = df['review_text'].apply(is_spam_repetition)
df['is_non_english'] = df['review_text'].apply(is_non_english)
df['is_emoji_spam'] = df['review_text'].apply(is_emoji_spam)

df['is_bad'] = (
    df['is_gibberish'] |
    df['is_spam_repetition'] |
    df['is_non_english'] |
    df['is_emoji_spam']
)

print(f"Total reviews: {len(df)}")
print(f"Gibberish: {df['is_gibberish'].sum()}")
print(f"Spam repetition: {df['is_spam_repetition'].sum()}")
print(f"Non-English: {df['is_non_english'].sum()}")
print(f"Emoji spam: {df['is_emoji_spam'].sum()}")
print(f"Total flagged: {df['is_bad'].sum()} ({df['is_bad'].mean()*100:.1f}%)")
print(f"Clean reviews remaining: {(~df['is_bad']).sum()}")

```

```

Total reviews:      6210
Gibberish:           360
Spam repetition:     182
Non-English:         0
Emoji spam:          0
Total flagged:       542 (8.7%)
Clean reviews remaining: 5668

```

```

In [10]: original_len = len(df)
df_clean = df[~df['is_bad']].copy().reset_index(drop=True)
print(f"Removed: {original_len - len(df_clean)} reviews")
print(f"Remaining: {len(df_clean)} reviews")

```

```

Removed: 542 reviews
Remaining: 5668 reviews

```

Before training we identify and remove problematic entries: gibberish, spam, non-English text, and emoji-only reviews. These would add noise without contributing useful signal.

5. Shared Text Preprocessing

All four models share the same cleaning pipeline:

- **Lowercasing**
- **Remove URLs and emails**
- **Remove non-letter characters** - punctuation and digits add noise for our bag-of-words models.
- **Remove stopwords (with exceptions)**
- **Normalise whitespace** - for consistent tokenization.

Each model then applies its own feature extraction (TF-IDF variants, or integer tokenization for the LSTM) on top of this cleaned text.

```
In [11]: stop_words = set(stopwords.words('english'))

sentiment_keepers = {
    'not', 'no', 'nor', 'never', 'neither', 'nobody', 'nothing', 'nowhere',
    'very', 'too', 'most', 'more', 'less', 'least', 'few',
    'but', 'however', 'although', 'only', 'just'
}
stop_words = stop_words - sentiment_keepers

domain_noise = {'app', 'whatsapp', 'apps'}
stop_words = stop_words | domain_noise

def clean_text(text):
    text = str(text).lower()
    text = re.sub(r'http\S+|www\S+', '', text)
    text = re.sub(r'\S+@\S+', '', text)
    text = re.sub(r'^a-z\s', '', text)
    text = re.sub(r'\s+', ' ', text).strip()
    text = ' '.join(w for w in text.split() if w not in stop_words)
    return text

df_clean['clean_text'] = df_clean['review_text'].apply(clean_text)

before = len(df_clean)
df_clean = df_clean[df_clean['clean_text'].str.len() > 0].reset_index(drop=True)
print(f"Dropped {before - len(df_clean)} empty reviews after cleaning")
print(f"Final dataset size: {len(df_clean)} reviews")
```

Dropped 11 empty reviews after cleaning
Final dataset size: 5657 reviews

```
In [12]: def sentiment_label(r):
    if r <= 2: return "negative"
    elif r == 3: return "neutral"
    else: return "positive"

df_clean['sentiment'] = df_clean['rating'].apply(sentiment_label)

print("Sentiment distribution:")
print(df_clean['sentiment'].value_counts())
```

```
Sentiment distribution:
sentiment
positive    3407
negative    1844
neutral      406
Name: count, dtype: int64
```

6. Target Variable and Train/Test Split

The target for all four models is the `rating` column (1–5 stars), directly addressing the task of predicting the assigned review score. The 80/20 train/test split is stratified on `rating` to preserve the class distribution in both sets. With `random_state=42` and a single split, all four models are trained and evaluated on identical data.

```
In [13]: df_train, df_test = train_test_split(
        df_clean,
        test_size=0.2,
        random_state=42,
        stratify=df_clean['rating']
    )

df_train = df_train.reset_index(drop=True)
df_test = df_test.reset_index(drop=True)

print(f"Train: {len(df_train)} reviews")
print(f"Test: {len(df_test)} reviews")
print("\nTrain rating distribution:")
print(df_train['rating'].value_counts().sort_index())
print("\nTest rating distribution:")
print(df_test['rating'].value_counts().sort_index())
```

```
Train: 4525 reviews
Test: 1132 reviews
```

```
Train rating distribution:
rating
1      1178
2       297
3       325
4       405
5     2320
Name: count, dtype: int64
```

```
Test rating distribution:
rating
1       295
2        74
3        81
4       101
5       581
Name: count, dtype: int64
```

7. Shared Design Decisions

- **Class weighting.** `class_weight='balanced'` or equivalent is applied to all four models. This is a deliberate trade-off: accuracy typically drops because the

model no longer leans on the majority class, but macro F1 should improve and all five classes are predicted rather than just rating 5.

- **Macro F1 as primary metric.** Accuracy will be dominated by rating 5 due to imbalance. The macro F1 treats all classes equally and better reflects performance on the harder minority classes.
- **Hyperparameter tuning via cross-validation** on the training set only, with the test set held out untouched until the final evaluation.
- **Why these models.** Logistic Regression and LinearSVC are strong baselines for high-dimensional sparse text and scale well to large vocabularies. LightGBM tests whether non-linear interactions help. The LSTM tests whether sequential context outperforms bag-of-words on this dataset size.

8. Model 1: Logistic Regression

Design choices. TF-IDF with `max_features=5000`, bigrams, and `min_df=5` for a compact feature space. Grid search over `C` and `penalty` (8 combinations, 3-fold CV) optimizing macro F1. `solver='saga'` supports both L1 and L2 with multiclass.

```
In [14]: X_train_lr = df_train['clean_text']
X_test_lr = df_test['clean_text']
y_train_lr = df_train['rating'].astype(int)
y_test_lr = df_test['rating'].astype(int)

tfidf_lr = TfidfVectorizer(max_features=5000, ngram_range=(1, 2), min_df=
X_train_lr_vec = tfidf_lr.fit_transform(X_train_lr)
X_test_lr_vec = tfidf_lr.transform(X_test_lr)

print(f"TF-IDF feature matrix shape: {X_train_lr_vec.shape}")

param_grid_lr = {
    'C': [0.01, 0.1, 1.0, 10.0],
    'penalty': ['l1', 'l2'],
}

grid_lr = GridSearchCV(
    LogisticRegression(
        max_iter=1000,
        class_weight='balanced',
        solver='saga'
    ),
    param_grid_lr,
    cv=3,
    scoring='f1_macro',
    n_jobs=1,
    verbose=1
)

grid_lr.fit(X_train_lr_vec, y_train_lr)
print(f"Best parameters: {grid_lr.best_params_}")
print(f"Best CV macro F1: {grid_lr.best_score_:.4f}")
```

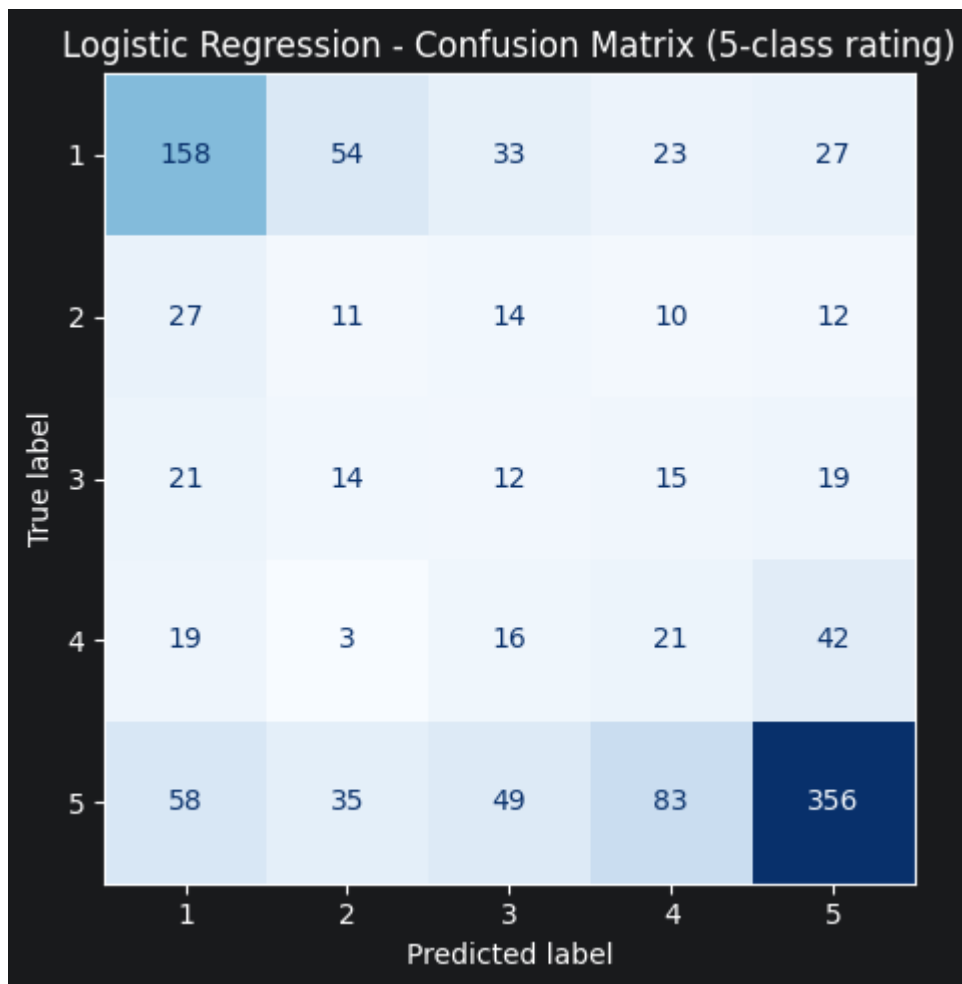
TF-IDF feature matrix shape: (4525, 1994)
Fitting 3 folds for each of 8 candidates, totalling 24 fits
Best parameters: {'C': 1.0, 'penalty': 'l2'}
Best CV macro F1: 0.3301

```
In [15]: lr_model = LogisticRegression(  
    **grid_lr.best_params_,  
    max_iter=1000,  
    class_weight='balanced',  
    solver='saga'  
)  
lr_model.fit(X_train_lr_vec, y_train_lr)  
  
y_pred_lr = lr_model.predict(X_test_lr_vec)  
  
target_names_lr = [f"Rating {i}" for i in range(1, 6)]  
print(f"\nAccuracy: {accuracy_score(y_test_lr, y_pred_lr):.4f}\n")  
print(classification_report(y_test_lr, y_pred_lr, target_names=target_names_lr))
```

Accuracy: 0.4929

	precision	recall	f1-score	support
Rating 1	0.56	0.54	0.55	295
Rating 2	0.09	0.15	0.12	74
Rating 3	0.10	0.15	0.12	81
Rating 4	0.14	0.21	0.17	101
Rating 5	0.78	0.61	0.69	581
accuracy			0.49	1132
macro avg	0.33	0.33	0.33	1132
weighted avg	0.57	0.49	0.53	1132

```
In [16]: cm_lr = confusion_matrix(y_test_lr, y_pred_lr, labels=[1, 2, 3, 4, 5])  
disp_lr = ConfusionMatrixDisplay(confusion_matrix=cm_lr, display_labels=[  
  
fig, ax = plt.subplots(figsize=(7, 5))  
disp_lr.plot(ax=ax, cmap="Blues", colorbar=False)  
ax.set_title("Logistic Regression - Confusion Matrix (5-class rating)")  
plt.tight_layout()  
plt.show()
```



LR - Results and discussion

The grid search selected $C=1.0$ with L2 penalty (CV macro F1: 0.330). Lower C values underfit the sparse TF-IDF features, higher values gave no gain.

The model reaches 49.3% accuracy and macro F1 of 0.33. Per-class results are reasonably even: ratings 2–4 reach F1 of 0.12–0.17, while ratings 1 and 5 are, as expected, the strongest with F1 = 0.55 and 0.69 respectively.

The main limitation is the 5,000-feature TF-IDF vocabulary, which likely caps how much signal the model can extract from the harder middle ratings. This is a limitation the SVM explores further.

9. Model 2: Support Vector Machine (LinearSVC)

Design choices. TF-IDF with `max_features=15,000`, bigrams, `sublinear_tf=True`, and `min_df=2`. This is a larger feature space than the LR-model to give the SVM-model more discriminative signal. C tuned with 5-fold CV over `[0.01, 0.1, 0.5, 1.0, 2.0, 5.0, 10.0]` using macro F1.

```
In [17]: X_train_svm = df_train['clean_text']
X_test_svm = df_test['clean_text']
y_train_svm = df_train['rating'].astype(int)
y_test_svm = df_test['rating'].astype(int)
```

```

svm_pipeline = Pipeline([
    ('tfidf', TfidfVectorizer(
        ngram_range=(1, 2),
        max_features=15_000,
        sublinear_tf=True,
        min_df=2,
    )),
    ('svm', LinearSVC(
        C=1.0,
        max_iter=2000,
        class_weight='balanced',
        random_state=42,
    ))
])

svm_pipeline.fit(X_train_svm, y_train_svm)
y_pred_svm = svm_pipeline.predict(X_test_svm)

print("SVM - Initial model (C=1.0)")
print(f"Accuracy: {accuracy_score(y_test_svm, y_pred_svm):.4f}\n")
print(classification_report(y_test_svm, y_pred_svm, zero_division=0))

```

SVM - Initial model (C=1.0)
Accuracy: 0.5539

	precision	recall	f1-score	support
1	0.54	0.60	0.57	295
2	0.08	0.07	0.08	74
3	0.08	0.06	0.07	81
4	0.15	0.15	0.15	101
5	0.73	0.73	0.73	581
accuracy			0.55	1132
macro avg	0.32	0.32	0.32	1132
weighted avg	0.54	0.55	0.55	1132

```

In [18]: C_values = [0.01, 0.1, 0.5, 1.0, 2.0, 5.0, 10.0]
mean_scores = []

for C in C_values:
    pipe = Pipeline([
        ('tfidf', TfidfVectorizer(
            ngram_range=(1, 2), max_features=15_000,
            sublinear_tf=True, min_df=2,
        )),
        ('svm', LinearSVC(C=C, max_iter=2000,
            class_weight='balanced', random_state=42))
    ])
    scores = cross_val_score(pipe, X_train_svm, y_train_svm, cv=5, scoring='f1_macro')
    mean_scores.append(scores.mean())
    print(f"C={C:6} → F1 macro: {scores.mean():.4f} ± {scores.std():.4f}")

best_C = C_values[np.argmax(mean_scores)]
print(f"\nBest C: {best_C}")

```

C=0.01 → F1 macro: 0.2764 ± 0.0114
 C=0.1 → F1 macro: 0.3354 ± 0.0077
 C=0.5 → F1 macro: 0.3243 ± 0.0096
 C=1.0 → F1 macro: 0.3216 ± 0.0082
 C=2.0 → F1 macro: 0.3146 ± 0.0085
 C=5.0 → F1 macro: 0.3061 ± 0.0103
 C=10.0 → F1 macro: 0.2999 ± 0.0070

Best C: 0.1

```
In [19]: final_svm = Pipeline([
    ('tfidf', TfidfVectorizer(
        ngram_range=(1, 2), max_features=15_000,
        sublinear_tf=True, min_df=2,
    )),
    ('svm', LinearSVC(
        C=best_C, max_iter=2000,
        class_weight='balanced', random_state=42,
    ))
])

final_svm.fit(X_train_svm, y_train_svm)
y_pred_svm_final = final_svm.predict(X_test_svm)

print(f"FINAL SVM (C={best_C})")
print(f"Accuracy: {accuracy_score(y_test_svm, y_pred_svm_final):.4f}\n")
print(classification_report(y_test_svm, y_pred_svm_final, zero_division=0))
```

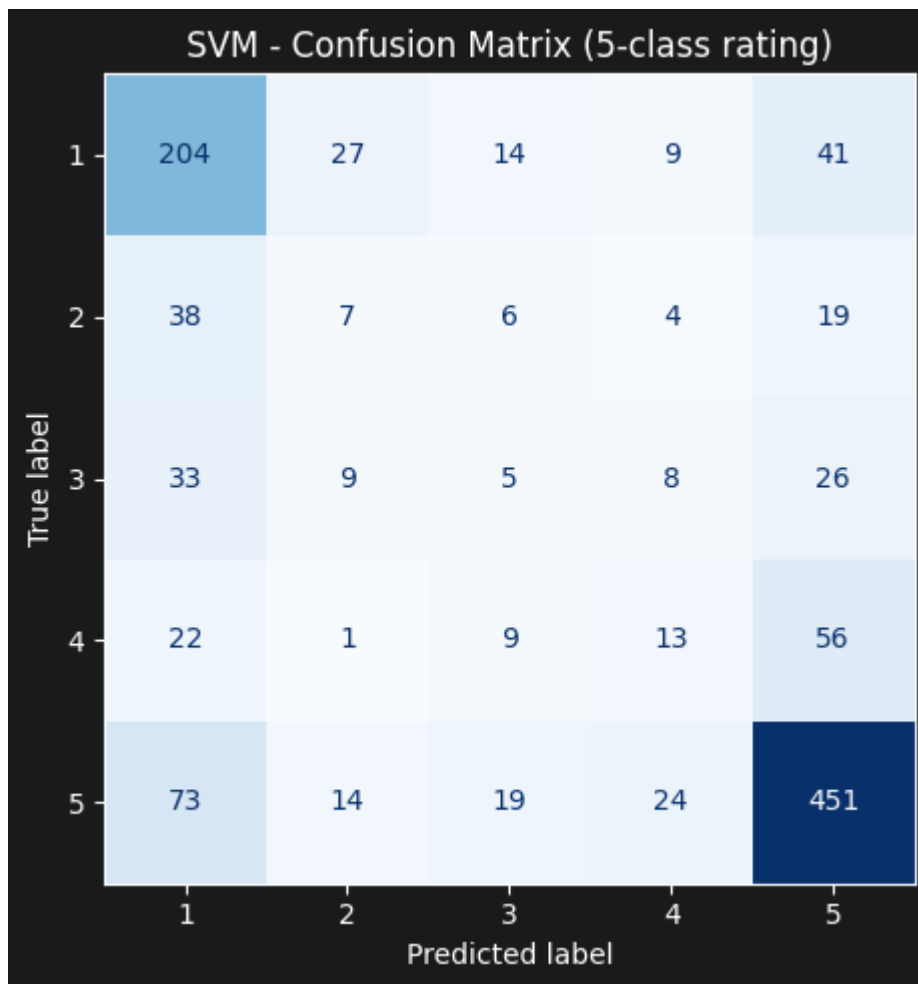
FINAL SVM (C=0.1)

Accuracy: 0.6007

	precision	recall	f1-score	support
1	0.55	0.69	0.61	295
2	0.12	0.09	0.11	74
3	0.09	0.06	0.07	81
4	0.22	0.13	0.16	101
5	0.76	0.78	0.77	581
accuracy			0.60	1132
macro avg	0.35	0.35	0.35	1132
weighted avg	0.57	0.60	0.58	1132

```
In [20]: labels = [1, 2, 3, 4, 5]
cm_svm = confusion_matrix(y_test_svm, y_pred_svm_final, labels=labels)
disp = ConfusionMatrixDisplay(confusion_matrix=cm_svm, display_labels=lab

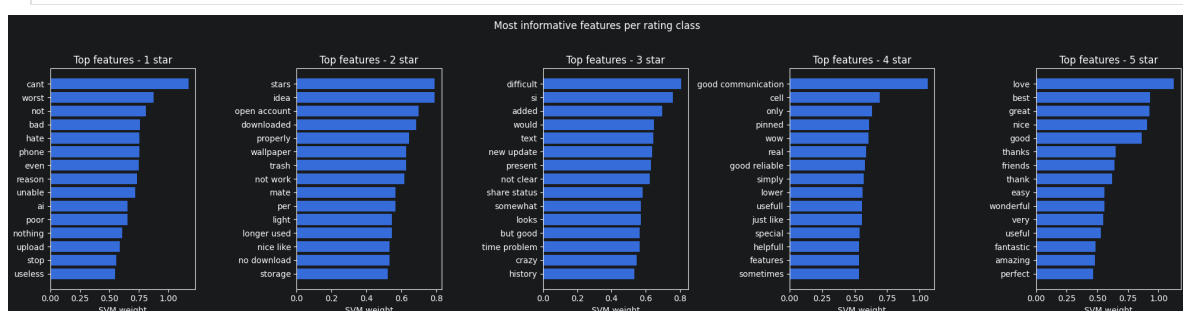
fig, ax = plt.subplots(figsize=(7, 5))
disp.plot(ax=ax, cmap='Blues', colorbar=False)
ax.set_title("SVM - Confusion Matrix (5-class rating)")
plt.tight_layout()
plt.show()
```



```
In [21]: feature_names = final_svm.named_steps['tfidf'].get_feature_names_out()
coef = final_svm.named_steps['svm'].coef_
```

```
fig, axes = plt.subplots(1, 5, figsize=(20, 5))
for i, (label, ax) in enumerate(zip(labels, axes)):
    top_idx = np.argsort(coef[i])[-15:]
    top_words = feature_names[top_idx]
    top_scores = coef[i][top_idx]
    ax.barh(top_words, top_scores)
    ax.set_title(f"Top features - {label} star")
    ax.set_xlabel("SVM weight")

plt.suptitle("Most informative features per rating class", y=1.02)
plt.tight_layout()
plt.show()
```



SVM - Results and discussion

CV selects $C=0.1$ (CV macro F1: 0.335), with scores declining steadily for higher C . The

default `C=1.0` overfits the TF-IDF space, and stronger regularization generalizes better. The tuned model reaches 60% accuracy and macro F1 of 0.35, up from 55% and 0.32 untuned.

Per-class spread is the widest of the four models. Rating 5 is strongest (F1 = 0.77) and rating 1 reaches 0.61, while ratings 2/3/4 sit at 0.11, 0.07, and 0.16 respectively. Tuning lifts rating 1 recall to 0.69, but ratings 2/3/4 stay the same. Stronger regularization helps the easiest minority class without rescuing the highly ambiguous middle ratings.

Feature importances explain the middle-class weakness: ratings 1 and 5 have sharp sentiment cues ("worst", "bad" vs "love", "great"), while 2/3/4 show ambiguous signals ("difficult", "somewhat", "good communication") which would be hard to resolve by the tuning itself.

10. Model 3: LightGBM

Design choices. TF-IDF with `max_features=20,000`, trigrams, `sublinear_tf=True`, and `max_df=0.9` to filter both rare and overly common terms. We initially ran a grid search over `n_estimators`, `learning_rate`, and `num_leaves` (8 combinations, 3-fold CV).

```
In [22]: X_train_lgb = df_train['clean_text']
X_test_lgb = df_test['clean_text']
y_train_lgb = df_train['rating']
y_test_lgb = df_test['rating']

tfidf_lgb = TfidfVectorizer(
    max_features=20000,
    ngram_range=(1, 3),
    min_df=3,
    max_df=0.9,
    sublinear_tf=True
)

X_train_lgb_vec = tfidf_lgb.fit_transform(X_train_lgb)
X_test_lgb_vec = tfidf_lgb.transform(X_test_lgb)

print(f"TF-IDF feature matrix shape: {X_train_lgb_vec.shape}")
```

TF-IDF feature matrix shape: (4525, 3891)

```
In [23]: model_gs = lgb.LGBMClassifier(
    class_weight='balanced',
    verbose=-1
)

param_grid = {
    "n_estimators": [100, 300],
    "learning_rate": [0.01, 0.1],
    "num_leaves": [31, 63],
}

grid = GridSearchCV(
    estimator=model_gs,
```

```

    param_grid=param_grid,
    cv=3,
    scoring="accuracy",
    verbose=1,
    n_jobs=1
)

grid.fit(X_train_lgb_vec, y_train_lgb)
print(f"Best parameters: {grid.best_params_}")
print(f"Best CV accuracy: {grid.best_score_:.4f}")

lgb_model = lgb.LGBMClassifier(
    **grid.best_params_,
    class_weight='balanced',
    verbose=-1
)
lgb_model.fit(X_train_lgb_vec, y_train_lgb)

y_pred_lgb = lgb_model.predict(X_test_lgb_vec)
print(f"\nTest Accuracy: {accuracy_score(y_test_lgb, y_pred_lgb):.4f}\n")
target_names = [f"Rating {i}" for i in range(1, 6)]
print(classification_report(y_test_lgb, y_pred_lgb, target_names=target_n

```

Fitting 3 folds for each of 8 candidates, totalling 24 fits

Best parameters: {'learning_rate': 0.1, 'n_estimators': 300, 'num_leaves': 63}

Best CV accuracy: 0.5271

Test Accuracy: 0.5353

	precision	recall	f1-score	support
Rating 1	0.51	0.54	0.52	295
Rating 2	0.06	0.04	0.05	74
Rating 3	0.08	0.06	0.07	81
Rating 4	0.14	0.12	0.13	101
Rating 5	0.69	0.74	0.71	581
accuracy			0.54	1132
macro avg	0.30	0.30	0.30	1132
weighted avg	0.51	0.54	0.52	1132

```

In [24]: feature_names_lgb = tfidf_lgb.get_feature_names_out()
importance = pd.DataFrame({
    "feature": feature_names_lgb,
    "importance": lgb_model.feature_importances_
}).sort_values(by="importance", ascending=False)

print("Top 20 most important features:")
print(importance.head(20).to_string(index=False))

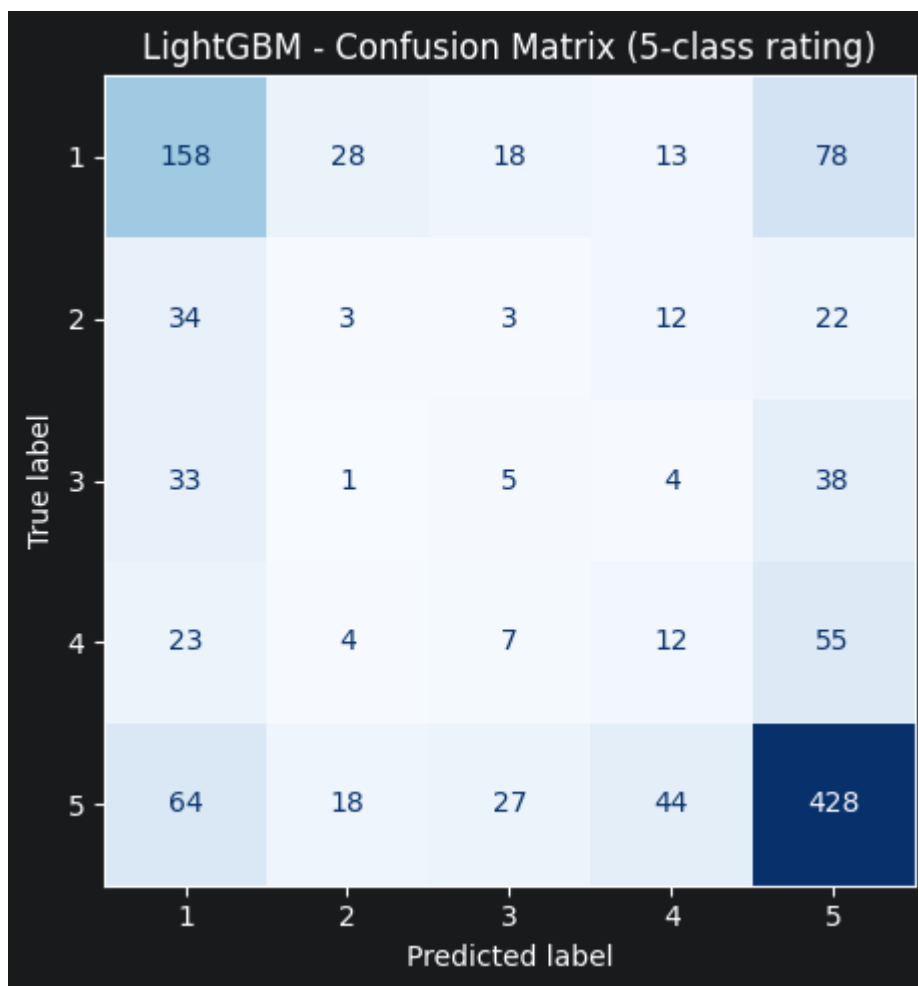
```

Top 20 most important features:

feature	importance
not	4340
good	3816
very	2987
please	2636
account	2112
but	2036
status	1715
use	1636
update	1593
cant	1581
love	1396
like	1278
nice	1270
chat	1202
problem	1192
best	1161
no	1134
new	1113
download	1107
open	1085

```
In [25]: cm_lgb = confusion_matrix(y_test_lgb, y_pred_lgb, labels=[1, 2, 3, 4, 5])
disp_lgb = ConfusionMatrixDisplay(confusion_matrix=cm_lgb, display_labels

fig, ax = plt.subplots(figsize=(7, 5))
disp_lgb.plot(ax=ax, cmap='Blues', colorbar=False)
ax.set_title("LightGBM - Confusion Matrix (5-class rating)")
plt.tight_layout()
plt.show()
```



LGBM - Results and discussion

Grid search selected `learning_rate=0.1` , `n_estimators=300` , and `num_leaves=63` (CV accuracy: 0.527). The more expressive trees (63 leaves vs. the initial 31) help the model use the upweighted minority signal.

The model reaches 54% accuracy and macro F1 of 0.30. Per-class spread is reasonable: ratings 2/3/4 reach F1 of 0.05/0.07/0.13, rating 5 has recall 0.74, and rating 1 stays the strongest minority class with F1 = 0.52.

Despite the trigram vocabulary, every top-20 feature is a unigram ("good", "not", "love", "cant"). No bigrams or trigrams rank highly, suggesting the dataset is too small for longer n-grams to accumulate enough signal. This also explains why LightGBM does not outperform the linear models here: tree ensembles need denser, more interactive features to show their advantage, which a small TF-IDF matrix does not provide.

To check whether a richer hyperparameter space would help, we then ran a randomized search over seven hyperparameters, adding `reg_alpha` , `reg_lambda` , `min_child_samples` , and `colsample_bytree` (20 sampled combinations, 3-fold CV). These four are the parameters most relevant for sparse high-dimensional features: L1/L2 regularization controls overfitting on the many near-zero TF-IDF weights, `min_child_samples` lets trees capture rare minority-class patterns, and column subsampling regularizes splits on the wide feature matrix. The expanded search roughly doubled runtime, but improved macro F1 by less than 0.01, so we kept the smaller grid.

The negative result from the expanded search shows the limitation of a small TF-IDF matrix. If the bottleneck was regularization or tree shape, the metrics should have moved.

11. Model 4: Bidirectional LSTM

Design choices.

- **Text augmentation** via synonym replacement for underrepresented classes, augmenting up to the count of rating 1 rather than rating 5 to avoid generating excessive synthetic samples.
- **Class weights** (inversely proportional to class frequency) during training, on top of augmentation.
- **A small Bidirectional LSTM** (32 units) with aggressive dropout (0.3–0.4). A larger architecture would overfit severely on ~6,000 reviews, because the embedding layer alone has 1.28M parameters.
- **EarlyStopping** on validation loss with patience 3.

11.1 Text Augmentation for Minority Classes

```
In [26]: df_trainval_lstm = df_train.copy()
df_test_lstm = df_test.copy()

print(f"Train+Val set: {len(df_trainval_lstm)} samples (will be augmented)
print(f"Test set: {len(df_test_lstm)} samples (original data only)"
print("\nClass distribution before augmentation:")
print(df_trainval_lstm['rating'].value_counts().sort_index())
```

```
Train+Val set: 4525 samples (will be augmented)
Test set: 1132 samples (original data only)
```

```
Class distribution before augmentation:
```

```
rating
1    1178
2     297
3     325
4     405
5    2320
```

```
Name: count, dtype: int64
```

```
In [27]: aug = naw.SynonymAug(aug_src='wordnet', aug_p=0.2)

count_per_class = df_trainval_lstm['rating'].value_counts().sort_index()
target_count = count_per_class[1]
print(f"Target count per class: {target_count}")

augmented_rows = []
for rating in [2, 3, 4]:
    class_df = df_trainval_lstm[df_trainval_lstm['rating'] == rating]
    current_count = len(class_df)
    needed = target_count - current_count
    if needed <= 0:
        continue
    print(f"Rating {rating}: {current_count} samples, generating {needed}
```

```

samples = class_df['clean_text'].tolist()
sources = random.choices(samples, k=needed)
augmented_texts = aug.augment(sources)
for text in augmented_texts:
    augmented_rows.append({'review_text': text, 'clean_text': text, '

aug_df = pd.DataFrame(augmented_rows)
df_trainval_balanced = pd.concat([df_trainval_lstm, aug_df], ignore_index
df_trainval_balanced = df_trainval_balanced.sample(frac=1, random_state=4

print("Train+Val distribution after augmentation:")
print(df_trainval_balanced['rating'].value_counts().sort_index())

```

Target count per class: 1178

Rating 2: 297 samples, generating 881 augmented samples

Rating 3: 325 samples, generating 853 augmented samples

Rating 4: 405 samples, generating 773 augmented samples

Train+Val distribution after augmentation:

rating

1 1178

2 1178

3 1178

4 1178

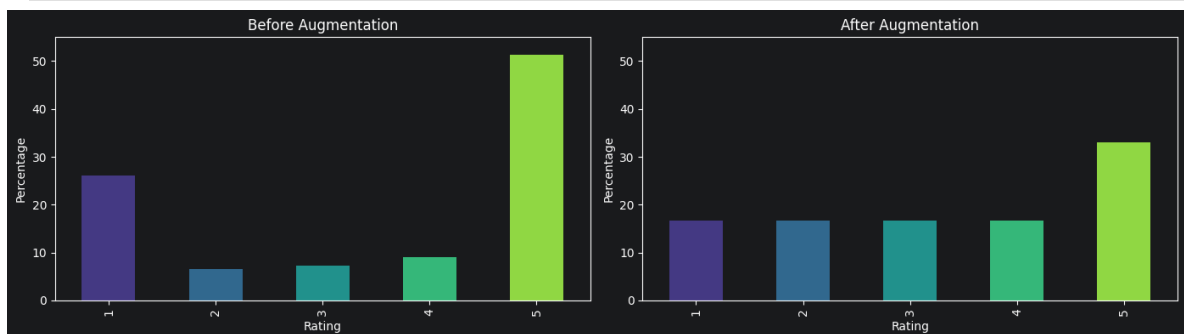
5 2320

Name: count, dtype: int64

```

In [28]: fig, axes = plt.subplots(1, 2, figsize=(14, 4))
for ax, data, title in zip(axes,
                            [df_trainval_lstm, df_trainval_balanced],
                            ['Before Augmentation', 'After Augmentation']):
    pct = data['rating'].value_counts(normalize=True).sort_index() * 100
    pct.plot(kind='bar', ax=ax, color=sns.color_palette('viridis', 5))
    ax.set_title(title)
    ax.set_ylabel('Percentage')
    ax.set_xlabel('Rating')
    ax.set_ylim(0, 55)
plt.tight_layout()
plt.show()

```



11.2 Tokenisation, Padding, and Data Splits

We convert each review into a sequence of integer tokens and pad/truncate to a fixed length (MAX_LEN=100) so every input to the LSTM has the same shape. The tokenizer is fit only on the training data to avoid data leakage.

We split the augmented training data into train (82%) and validation (18%) sets, giving an overall ~70/15/15 split.

```
In [29]: MAX_VOCAB = 20_000
MAX_LEN = 100
NUM_CLASSES = 5

tokenizer = Tokenizer(num_words=MAX_VOCAB, oov_token='<OOV>')
tokenizer.fit_on_texts(df_trainval_balanced['clean_text'])

def tokenize_and_pad(texts):
    seqs = tokenizer.texts_to_sequences(texts)
    return pad_sequences(seqs, maxlen=MAX_LEN, padding='post', truncating

X_trainval = tokenize_and_pad(df_trainval_balanced['clean_text'])
y_trainval = df_trainval_balanced['rating'].values - 1

X_test_final = tokenize_and_pad(df_test_lstm['clean_text'])
y_test_final = df_test_lstm['rating'].values - 1

X_train_lstm, X_val_lstm, y_train_int, y_val_int = train_test_split(
    X_trainval, y_trainval, test_size=0.18, random_state=42, stratify=y_t
)

y_train_lstm = to_categorical(y_train_int, NUM_CLASSES)
y_val_lstm = to_categorical(y_val_int, NUM_CLASSES)

print(f"Train: {X_train_lstm.shape[0]} Val: {X_val_lstm.shape[0]} Test:

Train: 5766 Val: 1266 Test: 1132
```

```
In [30]: class_weights = compute_class_weight(
    class_weight='balanced',
    classes=np.arange(NUM_CLASSES),
    y=y_train_int
)
class_weight_dict = dict(enumerate(class_weights))

print("Class weights:")
for cls, weight in class_weight_dict.items():
    print(f" Rating {cls + 1}: {weight:.3f}")
```

```
Class weights:
Rating 1: 1.194
Rating 2: 1.194
Rating 3: 1.194
Rating 4: 1.194
Rating 5: 0.606
```

11.3 - Architecture

- **Embedding (64-dim)** - dense vector per word.
- **SpatialDropout1D (0.3)** - drops entire embedding channels, which regularizes sequence models better than standard dropout.
- **Bidirectional LSTM (32 units)** - reads context in both directions.
- **Dense (16, ReLU) + Dropout (0.4)** - small head with high dropout.
- **Dense (5, softmax)** - output over 5 rating classes.

```
In [31]: EMBEDDING_DIM = 64

lstm_model = Sequential([
    Embedding(input_dim=MAX_VOCAB,
              output_dim=EMBEDDING_DIM,
              input_length=MAX_LEN),
    SpatialDropout1D(0.3),
    Bidirectional(LSTM(32, dropout=0.3, recurrent_dropout=0.3)),
    Dense(16, activation='relu'),
    Dropout(0.4),
    Dense(NUM_CLASSES, activation='softmax')
])

lstm_model.compile(
    loss='categorical_crossentropy',
    optimizer='adam',
    metrics=['accuracy']
)

lstm_model.build(input_shape=(None, MAX_LEN))
lstm_model.summary()
```

E0000 00:00:1777472932.319948 32897 cuda_platform.cc:52] failed call to cuInit: INTERNAL: CUDA error: Failed call to cuInit: UNKNOWN ERROR (303)
Model: "sequential"

Layer (type)	Output Shape	
embedding (Embedding)	(None , 100, 64)	1,2
spatial_dropout1d (SpatialDropout1D)	(None , 100, 64)	
bidirectional (Bidirectional)	(None , 64)	
dense (Dense)	(None , 16)	
dropout (Dropout)	(None , 16)	
dense_1 (Dense)	(None , 5)	

Total params: 1,305,957 (4.98 MB)

Trainable params: 1,305,957 (4.98 MB)

Non-trainable params: 0 (0.00 B)

10.4 Training


```
In [32]: early_stop = EarlyStopping(
    monitor='val_loss',
    patience=3,
    restore_best_weights=True,
    verbose=1
)

history = lstm_model.fit(
    X_train_lstm, y_train_lstm,
    epochs=20,
    batch_size=32,
    validation_data=(X_val_lstm, y_val_lstm),
    class_weight=class_weight_dict,
    callbacks=[early_stop],
    verbose=1
)
```

```
Epoch 1/20
181/181 ————— 16s 70ms/step - accuracy: 0.3229 - loss: 1.56
56 - val_accuracy: 0.4123 - val_loss: 1.4568
Epoch 2/20
181/181 ————— 11s 62ms/step - accuracy: 0.4095 - loss: 1.44
67 - val_accuracy: 0.4660 - val_loss: 1.3128
Epoch 3/20
181/181 ————— 11s 60ms/step - accuracy: 0.4835 - loss: 1.29
51 - val_accuracy: 0.5632 - val_loss: 1.2143
Epoch 4/20
181/181 ————— 11s 59ms/step - accuracy: 0.5852 - loss: 1.09
98 - val_accuracy: 0.5956 - val_loss: 1.1137
Epoch 5/20
181/181 ————— 12s 64ms/step - accuracy: 0.6615 - loss: 0.91
85 - val_accuracy: 0.6193 - val_loss: 1.0498
Epoch 6/20
181/181 ————— 12s 67ms/step - accuracy: 0.7133 - loss: 0.78
86 - val_accuracy: 0.6390 - val_loss: 1.0332
Epoch 7/20
181/181 ————— 12s 66ms/step - accuracy: 0.7470 - loss: 0.71
20 - val_accuracy: 0.6509 - val_loss: 1.0281
Epoch 8/20
181/181 ————— 13s 72ms/step - accuracy: 0.7735 - loss: 0.64
57 - val_accuracy: 0.6532 - val_loss: 1.0312
Epoch 9/20
181/181 ————— 13s 71ms/step - accuracy: 0.7764 - loss: 0.60
87 - val_accuracy: 0.6588 - val_loss: 1.0703
Epoch 10/20
181/181 ————— 13s 71ms/step - accuracy: 0.8006 - loss: 0.56
02 - val_accuracy: 0.6564 - val_loss: 1.0962
Epoch 10: early stopping
Restoring model weights from the end of the best epoch: 7.
```

10.5 Evaluation

```
In [33]: fig, axes = plt.subplots(1, 2, figsize=(14, 5))
axes[0].plot(history.history['loss'], label='Train Loss')
axes[0].plot(history.history['val_loss'], label='Val Loss')
axes[0].set_title('Loss over Epochs')
axes[0].set_xlabel('Epoch'); axes[0].set_ylabel('Loss'); axes[0].legend()

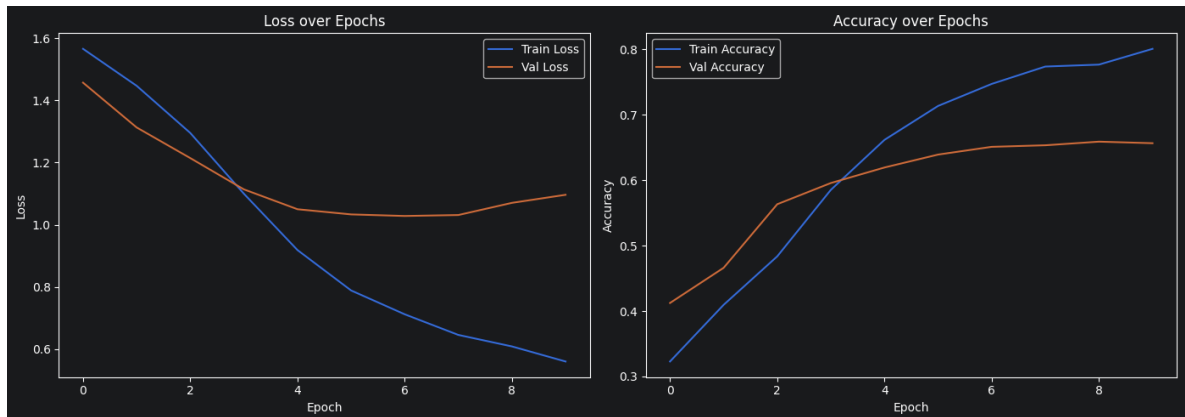
axes[1].plot(history.history['accuracy'], label='Train Accuracy')
```

```

axes[1].plot(history.history['val_accuracy'], label='Val Accuracy')
axes[1].set_title('Accuracy over Epochs')
axes[1].set_xlabel('Epoch'); axes[1].set_ylabel('Accuracy'); axes[1].legende

plt.tight_layout()
plt.show()

```



```

In [34]: y_score_lstm = lstm_model.predict(X_test_final)
y_pred_lstm = np.argmax(y_score_lstm, axis=1)

print(f"Test Accuracy: {accuracy_score(y_test_final, y_pred_lstm):.4f}")
target_names = [f"Rating {i}" for i in range(1, NUM_CLASSES + 1)]
print(classification_report(y_test_final, y_pred_lstm, target_names=target_names))

```

36/36 ————— 2s 29ms/step

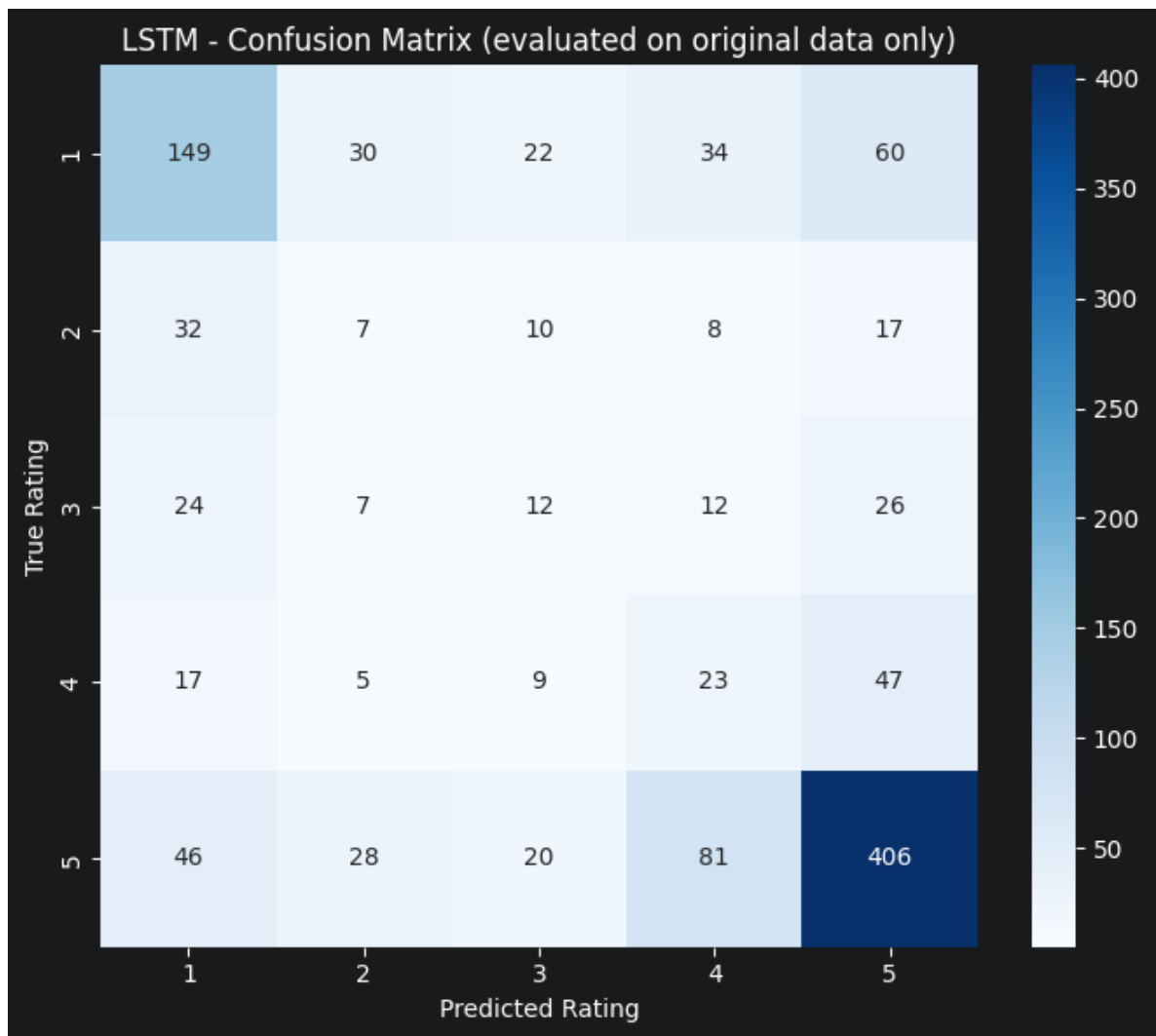
Test Accuracy: 0.5274

	precision	recall	f1-score	support
Rating 1	0.56	0.51	0.53	295
Rating 2	0.09	0.09	0.09	74
Rating 3	0.16	0.15	0.16	81
Rating 4	0.15	0.23	0.18	101
Rating 5	0.73	0.70	0.71	581
accuracy			0.53	1132
macro avg	0.34	0.33	0.33	1132
weighted avg	0.55	0.53	0.54	1132

```

In [35]: cm_lstm = confusion_matrix(y_test_final, y_pred_lstm)
plt.figure(figsize=(7, 6))
sns.heatmap(cm_lstm, annot=True, fmt='d', cmap='Blues',
            xticklabels=range(1, NUM_CLASSES + 1),
            yticklabels=range(1, NUM_CLASSES + 1))
plt.title('LSTM - Confusion Matrix (evaluated on original data only)')
plt.xlabel('Predicted Rating')
plt.ylabel('True Rating')
plt.tight_layout()
plt.show()

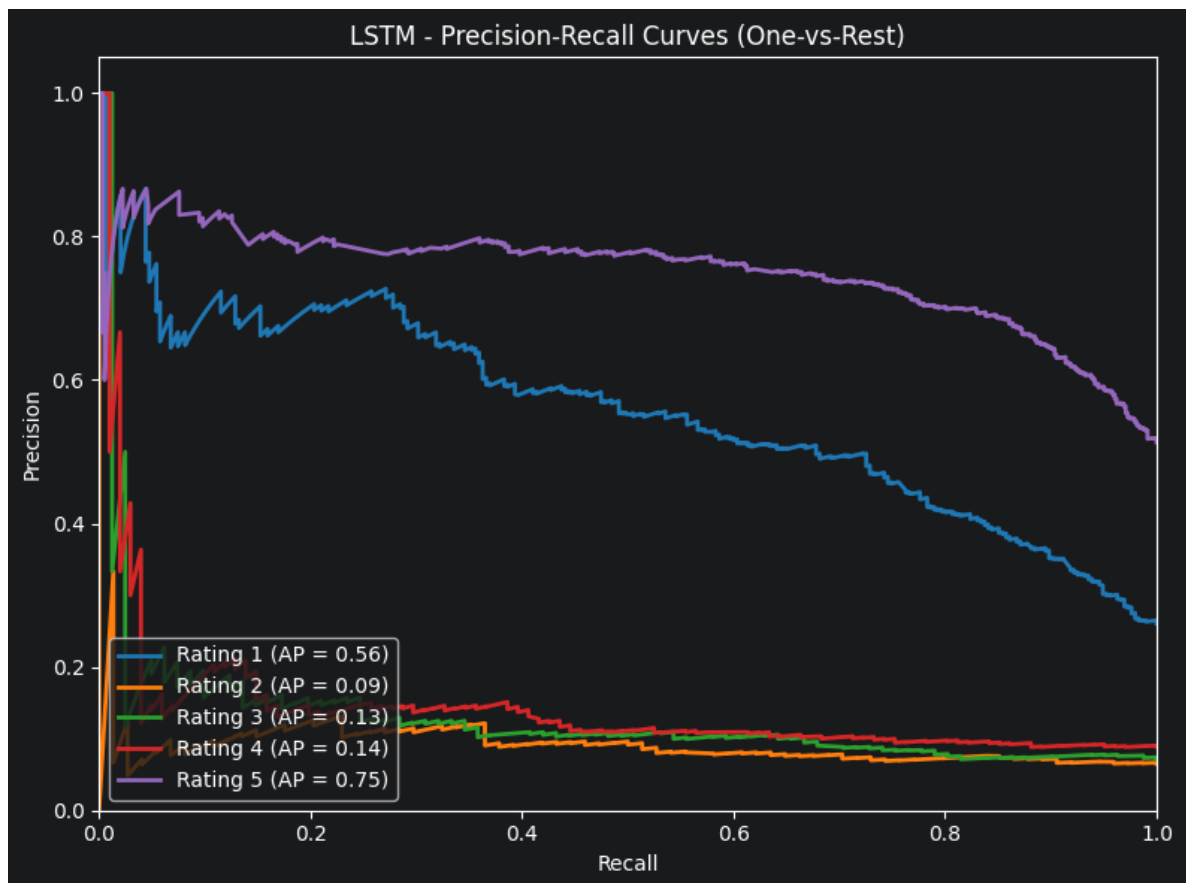
```



```
In [36]: y_test_bin = label_binarize(y_test_final, classes=range(NUM_CLASSES))
        colors = ['#1f77b4', '#ff7f0e', '#2ca02c', '#d62728', '#9467bd']

        fig, ax = plt.subplots(figsize=(8, 6))
        for i in range(NUM_CLASSES):
            precision, recall, _ = precision_recall_curve(y_test_bin[:, i], y_score_lstm[:, i])
            ap = average_precision_score(y_test_bin[:, i], y_score_lstm[:, i])
            ax.plot(recall, precision, color=colors[i], lw=2,
                    label=f'Rating {i + 1} (AP = {ap:.2f})')

        ax.set_xlabel('Recall')
        ax.set_ylabel('Precision')
        ax.set_title('LSTM - Precision-Recall Curves (One-vs-Rest)')
        ax.legend(loc='lower left')
        ax.set_xlim([0.0, 1.0])
        ax.set_ylim([0.0, 1.05])
        plt.tight_layout()
        plt.show()
```



LSTM - Results and discussion

The LSTM reaches 53% accuracy and macro F1 of 0.33. EarlyStopping restored weights from epoch 7, where the gap between training accuracy 75% and validation accuracy 65% is already visible. Overfitting begins from epoch 8 despite the regularization. This reflects the fundamental limitation of training a 1.28M-parameter network on ~6,000 reviews.

Per-class results show the most balanced spread of all four models: rating 5 at F1 = 0.71, rating 1 at 0.53, and ratings 2/3/4 at 0.09/0.16/0.18. Compared to the other models, the LSTM trades some rating 1 performance for more even coverage of the middle ratings. The combined effect of augmentation and class weighting redistributing the model's capacity.

We tested several variants (`MAX_LEN=150` , patience 5, augmenting up to rating 5's count) but macro F1 stayed at 0.32–0.33, so we kept the simpler configuration. This stability suggests the bottleneck is dataset size and architecture, not hyperparameters. Some likely improvements could be using pre-trained embeddings (GloVe, FastText), a pre-trained transformer (DistilBERT), or reducing to 3 classes to merge the ambiguous middle ratings. We concluded that these implementations are either beyond the scope of this assignment or don't address the task effectively.

12. Model Comparison and Summary

Model	Accuracy	Macro F1	Notes
Logistic Regression	0.493	0.33	Class weighted; C=1.0, L2 penalty found via grid search
LinearSVC (SVM)	0.601	0.35	Best macro F1; C=0.1 found via 5-fold CV
LightGBM	0.535	0.30	Class weighted; best params found via grid search
Bidirectional LSTM	0.527	0.33	Most balanced minority class coverage

Key findings. The SVM is the best overall (highest accuracy and macro F1). Linear models with TF-IDF are competitive, this is typical for small text datasets where TF-IDF already captures most of the signal and complex models cannot show their advantage. The LSTM matches LR on macro F1 (0.33) despite very different per-class behavior — the LSTM achieves more balanced minority-class coverage, while LR leans harder on the easier rating 1 and rating 5 classes. LightGBM is the weakest, reflecting that tree ensembles struggle to exploit sparse high-dimensional TF-IDF features when bigrams and trigrams contribute little signal.

Why macro F1 matters here. LR and the LSTM have the same macro F1 (0.33) despite quite different accuracy (0.493 vs. 0.527), because accuracy is dominated by rating 5 while macro F1 weights all classes equally. This confirms macro F1 as the more informative metric for this task.

Class imbalance is the dominant constraint. Across all four models, ratings 2 and 3 stay as the hardest classes to categorize ($F1 = 0.05\text{--}0.15$), even with class weighting and augmentation. The middle ratings are highly ambiguous in the data, many such reviews mix positive and negative language, and no amount of tuning recovers them.

Overfitting control across models. L2 regularization with tuned C (LR, SVM), tree complexity tuning (LightGBM), dropout and EarlyStopping (LSTM), plus the held-out test set and CV-based hyperparameter selection used everywhere.

Task 2: AI vs. Human Art Classifier

- **Dataset:** Art_shuffled/ - 539 AI-generated, 436 real art images (975 total).
- **Model:** EfficientNetB0 via transfer learning.

1. Data Analysis

The dataset has a mild 55/45 imbalance. A naive classifier always predicting "AI" would reach ~55% accuracy, so meaningful performance must clearly exceed this. With fewer than 1,000 images, training a deep CNN from scratch would overfit severely, this motivated our choice of transfer learning.

```
In [37]: ai_count    = len(os.listdir(os.path.join(dataset_path, 'AiArtData/')))
real_count = len(os.listdir(os.path.join(dataset_path, 'RealArt/')))

print(f"AI images:    {ai_count}")
print(f"Real images:  {real_count}")
print(f"Total:        {ai_count + real_count}")
print(f"Class ratio:   {ai_count / (ai_count + real_count):.2%} AI / {real_count / (ai_count + real_count):.2%} Real")
```

AI images: 559
Real images: 453
Total: 1012
Class ratio: 55.24% AI / 44.76% Real

2. Preprocessing and augmentation

All images are resized to 224×224 to match EfficientNetB0's expected input. To compensate for the small dataset, we apply light augmentation (rotation $\pm 15^\circ$, horizontal flip, zoom $\pm 10\%$) during training to increase variety and reduce overfitting. 20% of the data is held out for validation using `ImageDataGenerator`'s `subset` parameter, ensuring consistent splitting.

```
In [38]: IMG_SIZE    = (224, 224)
BATCH_SIZE = 32

train_gen = ImageDataGenerator(
    rotation_range=15,
    horizontal_flip=True,
    zoom_range=0.1,
    validation_split=0.2
)

train_data = train_gen.flow_from_directory(
    dataset_path,
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode='binary',
    subset='training',
    seed=42
)

val_data = train_gen.flow_from_directory(
    dataset_path,
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode='binary',
    subset='validation',
    seed=42
)
```

```
)

print("Class indices:", train_data.class_indices)
```

Found 781 images belonging to 2 classes.
Found 194 images belonging to 2 classes.
Class indices: {'AiArtData': 0, 'RealArt': 1}

3. Architecture - EfficientNetB0

We use EfficientNetB0 pretrained on ImageNet. EfficientNet's compound scaling gives strong performance per parameter, B0 is the smallest variant (suitable for our computational constraint), and transfer learning leverages ImageNet features that transfer well to art images. With <1,000 training samples, a smaller pretrained model is preferable to a larger one, the additional capacity of B3 or B7 would not be supportable.

The custom head on top of the frozen base:

- **GlobalAveragePooling2D** - reduces feature maps to a single vector per channel, avoiding the very large parameter count that flattening would produce.
- **Dense(128, ReLU)** - task-specific feature combinations.
- **Dropout(0.3)** - regularization in the head.
- **Dense(1, sigmoid)** - binary output (0 = AI, 1 = real).

```
In [39]: base_model = EfficientNetB0(weights='imagenet', include_top=False, input_shape=(224, 224, 3))
base_model.trainable = False

inputs = tf.keras.Input(shape=(224, 224, 3))
x = base_model(inputs, training=False)
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dense(128, activation='relu')(x)
x = layers.Dropout(0.3)(x)
outputs = layers.Dense(1, activation='sigmoid')(x)
model = models.Model(inputs, outputs)

model.summary()
```

Model: "functional_1"

Layer (type)	Output Shape	Param #
input_layer_2 (InputLayer)	(None, 224, 224, 3)	
efficientnetb0 (Functional)	(None, 7, 7, 1280)	4,213,668
global_average_pooling2d (GlobalAveragePooling2D)	(None, 1280)	
dense_2 (Dense)	(None, 128)	163,904
dropout_1 (Dropout)	(None, 128)	
dense_3 (Dense)	(None, 1)	128

Total params: 4,213,668 (16.07 MB)

Trainable params: 164,097 (641.00 KB)

Non-trainable params: 4,049,571 (15.45 MB)

4. Training in two phases

Phase 1 - frozen base. Only the head is trained, so the new layers can adapt to the task without disrupting pretrained ImageNet features. Adam at $\text{lr}=1\text{e-}3$ is appropriate since only the small head is updated. EarlyStopping on `val_loss` with patience 3 and `restore_best_weights=True`.

```
In [40]: early_stop_phase1 = EarlyStopping(  
        monitor='val_loss',  
        patience=3,  
        restore_best_weights=True,  
        verbose=1  
    )  
  
    model.compile(  
        optimizer=tf.keras.optimizers.Adam(1e-3),  
        loss='binary_crossentropy',  
        metrics=['accuracy']  
    )  
  
    history_phase1 = model.fit(  
        train_data,  
        validation_data=val_data,  
        epochs=20,  
        callbacks=[early_stop_phase1]  
    )
```

Epoch 1/20

I0000 00:00:1777473066.174828 32897 generator_dataset_op.cc:213] Memory patch applied: M_TRIM_THRESHOLD=128 kb was set.

25/25 ————— 72s 2s/step - accuracy: 0.6927 - loss: 0.5761 -
val_accuracy: 0.7320 - val_loss: 0.5184

Epoch 2/20

25/25 ————— 55s 2s/step - accuracy: 0.8348 - loss: 0.3809 -
val_accuracy: 0.7577 - val_loss: 0.4792

Epoch 3/20

25/25 ————— 49s 2s/step - accuracy: 0.8720 - loss: 0.3249 -
val_accuracy: 0.7629 - val_loss: 0.4838

Epoch 4/20

25/25 ————— 52s 2s/step - accuracy: 0.8963 - loss: 0.2631 -
val_accuracy: 0.7629 - val_loss: 0.4820

Epoch 5/20

25/25 ————— 50s 2s/step - accuracy: 0.9245 - loss: 0.2215 -
val_accuracy: 0.7320 - val_loss: 0.5519

Epoch 5: early stopping

Restoring model weights from the end of the best epoch: 2.

Phase 2 - fine-tuning. We unfreeze the last 20 layers of EfficientNetB0 to adapt higher-level features toward AI-vs-human cues (brushstroke regularity, noise distribution, stylistic consistency etc.). Earlier layers capture transferable low-level features and stay frozen. Adam at $\text{lr}=1\text{e-}5$ - a much lower rate, since aggressive updates would destroy the pretrained representations. EarlyStopping again with patience 3.


```
In [41]: base_model.trainable = True
for layer in base_model.layers[:-20]:
    layer.trainable = False

early_stop_phase2 = EarlyStopping(
    monitor='val_loss',
    patience=3,
    restore_best_weights=True,
    verbose=1
)

model.compile(
    optimizer=tf.keras.optimizers.Adam(1e-5),
    loss='binary_crossentropy',
    metrics=['accuracy']
)

history_phase2 = model.fit(
    train_data,
    validation_data=val_data,
    epochs=20,
    callbacks=[early_stop_phase2]
)

model.save('art_classifier.keras')
print("Model saved to art_classifier.keras")
```

Epoch 1/20
25/25 ————— 57s 2s/step - accuracy: 0.8502 - loss: 0.4002 - val_accuracy: 0.7526 - val_loss: 0.4564
Epoch 2/20
25/25 ————— 49s 2s/step - accuracy: 0.8604 - loss: 0.3707 - val_accuracy: 0.7680 - val_loss: 0.4813
Epoch 3/20
25/25 ————— 48s 2s/step - accuracy: 0.8604 - loss: 0.3811 - val_accuracy: 0.7423 - val_loss: 0.4793
Epoch 4/20
25/25 ————— 48s 2s/step - accuracy: 0.8592 - loss: 0.3702 - val_accuracy: 0.7423 - val_loss: 0.4841
Epoch 4: early stopping
Restoring model weights from the end of the best epoch: 1.
Model saved to art_classifier.keras

Training History Visualisation

```
In [42]: def plot_history(history, title_prefix):
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))

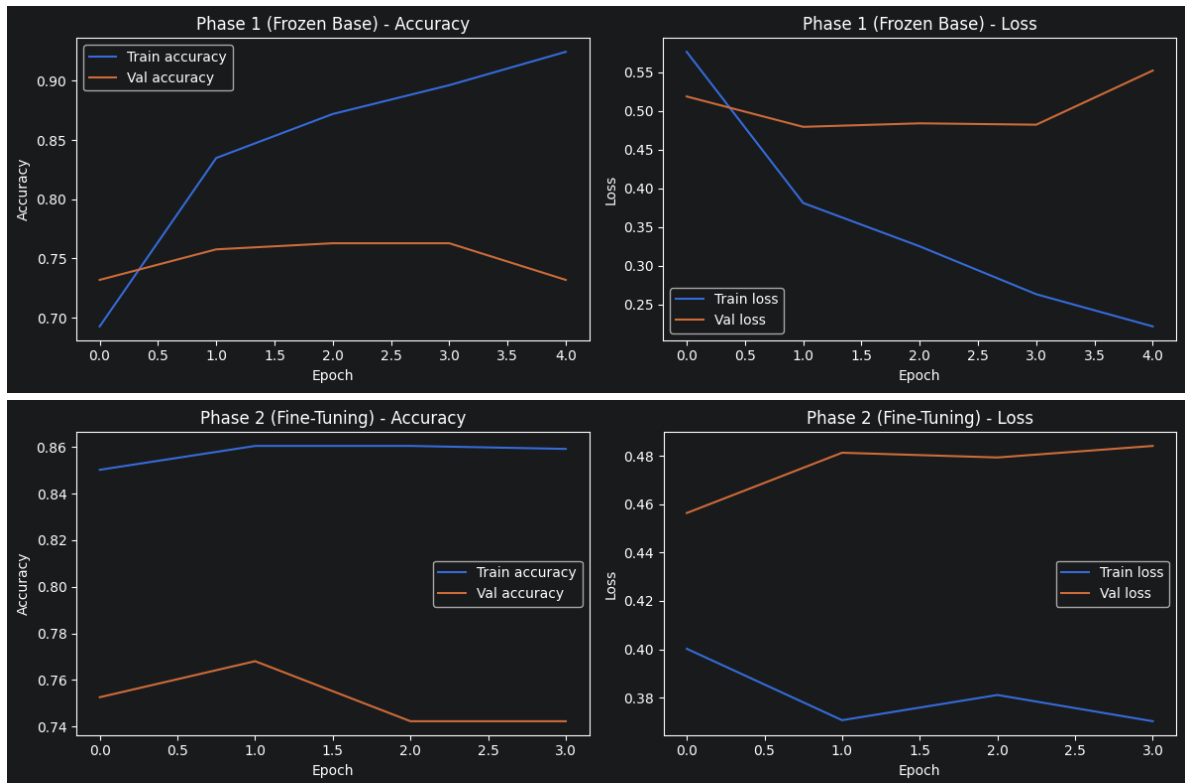
ax1.plot(history.history['accuracy'], label='Train accuracy')
ax1.plot(history.history['val_accuracy'], label='Val accuracy')
ax1.set_title(f'{title_prefix} - Accuracy')
ax1.set_xlabel('Epoch')
ax1.set_ylabel('Accuracy')
ax1.legend()

ax2.plot(history.history['loss'], label='Train loss')
ax2.plot(history.history['val_loss'], label='Val loss')
ax2.set_title(f'{title_prefix} - Loss')
```

```
ax2.set_xlabel('Epoch')
ax2.set_ylabel('Loss')
ax2.legend()
```

```
plt.tight_layout()
plt.show()
```

```
plot_history(history_phase1, 'Phase 1 (Frozen Base)')
plot_history(history_phase2, 'Phase 2 (Fine-Tuning)')
```



```
In [43]: eval_gen = ImageDataGenerator(validation_split=0.2)

eval_data = eval_gen.flow_from_directory(
    dataset_path,
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode='binary',
    subset='validation',
    seed=42,
    shuffle=False
)

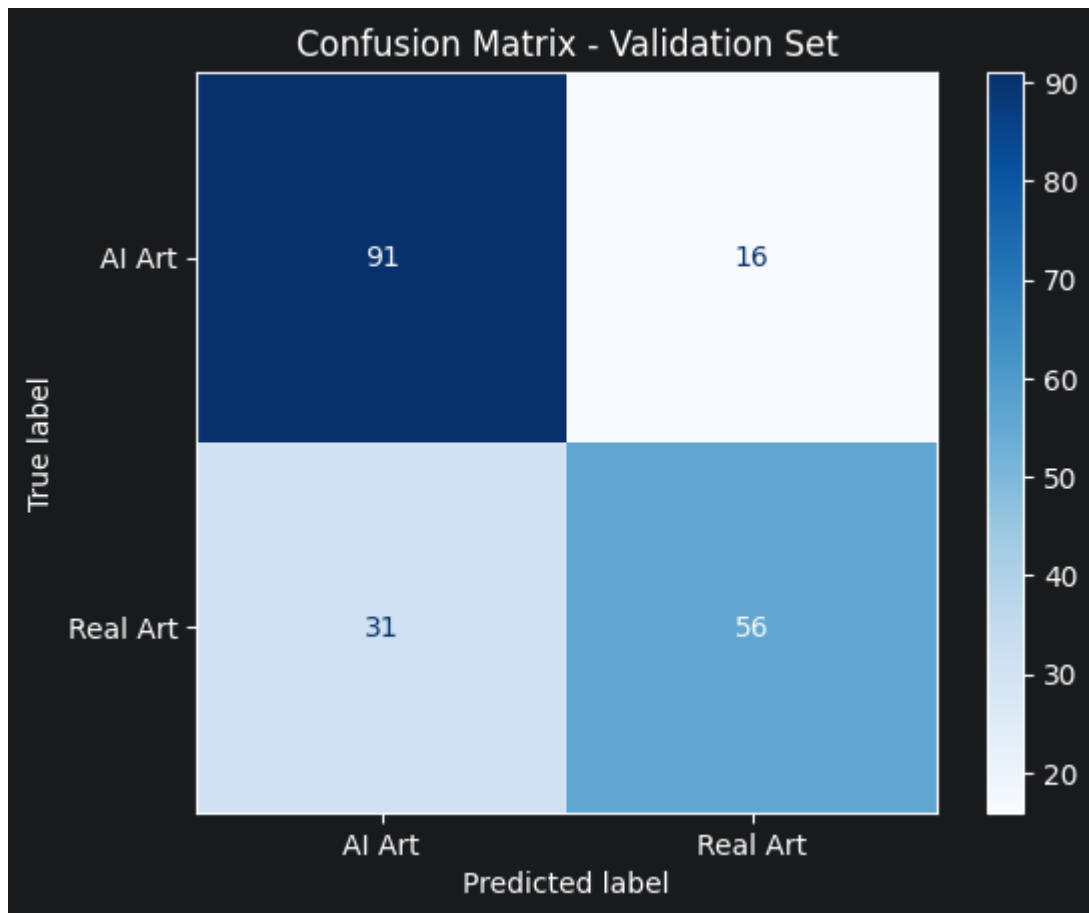
y_pred_probs = model.predict(eval_data, verbose=0)
y_pred = (y_pred_probs > 0.5).astype(int).flatten()
y_true = eval_data.classes

print(classification_report(y_true, y_pred, target_names=['AI Art', 'Real

cm = confusion_matrix(y_true, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['AI Ar
disp.plot(cmap='Blues')
plt.title('Confusion Matrix - Validation Set')
plt.show()
```

Found 194 images belonging to 2 classes.

	precision	recall	f1-score	support
AI Art	0.75	0.85	0.79	107
Real Art	0.78	0.64	0.70	87
accuracy			0.76	194
macro avg	0.76	0.75	0.75	194
weighted avg	0.76	0.76	0.75	194



Summary and discussion

Results

The model achieves 76% validation accuracy on the held-out validation set. The small gap between this and the final evaluation score reflects normal variation between the shuffled training generator and the fixed evaluation set. Given the small dataset size and the inherent difficulty of the task, 76% is a reasonable result.

Class-wise performance

The classification report shows AI Art recall of 0.85 substantially exceeds Real Art recall of 0.64, consistent with the mild class imbalance: 55% AI, 45% real. The model learns a slight bias toward the majority class.

Overfitting analysis

The training curves show a moderate overfitting pattern: training accuracy climbs to ~92% before EarlyStopping intervenes, while validation accuracy plateaus around 75–76%. This gap is to be expected for a dataset of this size. EarlyStopping triggered early in both phases, after epoch 5 in Phase 1 and epoch 4 in Phase 2, preventing further divergence between training and validation performance.

The mitigation strategies applied: Dropout (0.3), Data augmentation and EarlyStopping kept the train/val gap moderate.

Further improvements that were not pursued due to resource constraints:

- More aggressive augmentation (color jitter, random cropping)
- L2 weight regularization in the dense layer
- A larger dataset would be the most impactful fix

Fine-tuning assessment

Phase 2 fine-tuning produced only marginal gains over Phase 1, with EarlyStopping restoring weights from epoch 1 of Phase 2. Effectively the model improved very little during fine-tuning. This suggests the frozen ImageNet features were already well-suited to the task, and that the primary bottleneck is dataset size rather than feature quality.

Task 2b: Classifying 5 New Images

For Task 2b we test the trained model on five AI-generated images stored locally in `task2_new_images/` with subfolders `AiArtData/` and `RealArt/`.

The trained model is loaded from the local file `art_classifier.keras` produced in Phase 2 above.

Each image is resized to 224×224 and loaded as RGB, matching the preprocessing used during training.

No manual pixel rescaling is applied. EfficientNetB0 includes its own internal preprocessing when called with pretrained ImageNet weights.

The model outputs a sigmoid score:

- < 0.5 is predicted AI-generated
- ≥ 0.5 is predicted human-made

```
In [44]: MODEL_PATH = Path('art_classifier.keras')
NEW_IMAGE_DIR = Path('task2_new_images')
SUPPORTED_EXT = {'.jpg', '.jpeg', '.png', '.webp', '.bmp'}

assert MODEL_PATH.exists(), f"Model not found: {MODEL_PATH}"
assert NEW_IMAGE_DIR.exists(), f"Image folder not found: {NEW_IMAGE_DIR}"

if 'model' not in dir() or model is None:
    model = load_model(MODEL_PATH, compile=False)
    print('Loaded model from', MODEL_PATH)
else:
```

```

        print('Using model already in memory.')

new_image_paths = sorted(
    p for p in NEW_IMAGE_DIR.rglob('*')
    if p.is_file() and p.suffix.lower() in SUPPORTED_EXT
)

print(f'Found {len(new_image_paths)} images in {NEW_IMAGE_DIR}')
for p in new_image_paths:
    print(f'    {p.parent.name}/{p.name}')

```

Using model already in memory.

Found 5 images in task2_new_images

AiArtData/Screenshot 2026-04-22 at 13.21.45.png

AiArtData/Screenshot 2026-04-22 at 13.21.56.png

AiArtData/aiart2.png

RealArt/IMG_7674.jpeg

RealArt/IMG_8190.jpeg

```

In [45]: class_indices = {'AiArtData': 0, 'RealArt': 1}
index_to_class = {v: k for k, v in class_indices.items()}

def prepare_image(path):
    img = load_img(path, target_size=IMG_SIZE)
    array = img_to_array(img)
    return array

image_batch = np.stack([prepare_image(p) for p in new_image_paths])
prob_real = model.predict(image_batch, verbose=0).reshape(-1)
predicted_indices = (prob_real >= 0.5).astype(int)

true_labels = [p.parent.name for p in new_image_paths]
true_indices = [class_indices[l] for l in true_labels]

results = pd.DataFrame({
    'image': [p.name for p in new_image_paths],
    'true_label': true_labels,
    'probability_real_art': prob_real,
    'predicted_label': [index_to_class[i] for i in predicted_indices]
})
results['correct'] = results['true_label'] == results['predicted_label']
results['confidence'] = np.where(
    predicted_indices == 1,
    results['probability_real_art'],
    1 - results['probability_real_art']
)

accuracy = accuracy_score(true_indices, predicted_indices)
print(f'Accuracy on {len(new_image_paths)} new images: {accuracy:.2%}')
display(results)

print(classification_report(
    true_indices, predicted_indices,
    labels=[0, 1],
    target_names=[index_to_class[0], index_to_class[1]],
    zero_division=0,
))

```

WARNING:tensorflow:5 out of the last 44 calls to <function TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_distributed at 0x7b865566fa60> triggered tf.function retracing. Tracing is expensive and the excessive number of tracings could be due to (1) creating @tf.function repeatedly in a loop, (2) passing tensors with different shapes, (3) passing Python objects instead of tensors. For (1), please define your @tf.function outside of the loop. For (2), @tf.function has reduce_retracing=True option that can avoid unnecessary retracing. For (3), please refer to https://www.tensorflow.org/guide/function#controlling_retracing and https://www.tensorflow.org/api_docs/python/tf/function for more details.

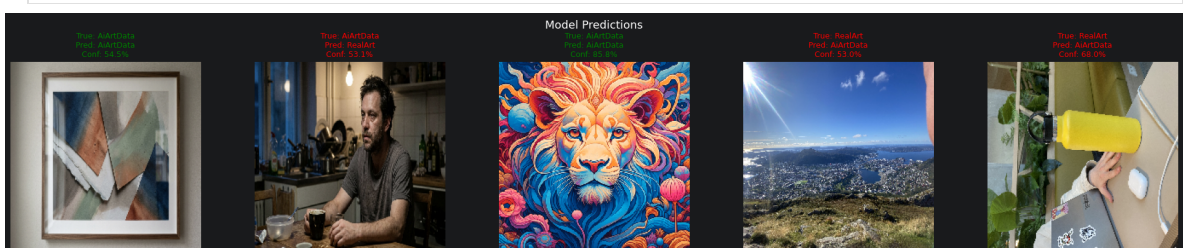
Accuracy on 5 new images: 40.00%

	image	true_label	probability_real_art	predicted_label	correct	confidence
0	Screenshot 2026-04-22 at 13.21.45.png	AiArtData	0.455122	AiArtData	True	0.544878
1	Screenshot 2026-04-22 at 13.21.56.png	AiArtData	0.531387	RealArt	False	0.531387
2	aiart2.png	AiArtData	0.141655	AiArtData	True	0.858345
3	IMG_7674.jpeg	RealArt	0.470150	AiArtData	False	0.529850
4	IMG_8190.jpeg	RealArt	0.320340	AiArtData	False	0.679660
		precision	recall	f1-score	support	
	AiArtData	0.50	0.67	0.57	3	
	RealArt	0.00	0.00	0.00	2	
	accuracy			0.40	5	
	macro avg	0.25	0.33	0.29	5	
	weighted avg	0.30	0.40	0.34	5	

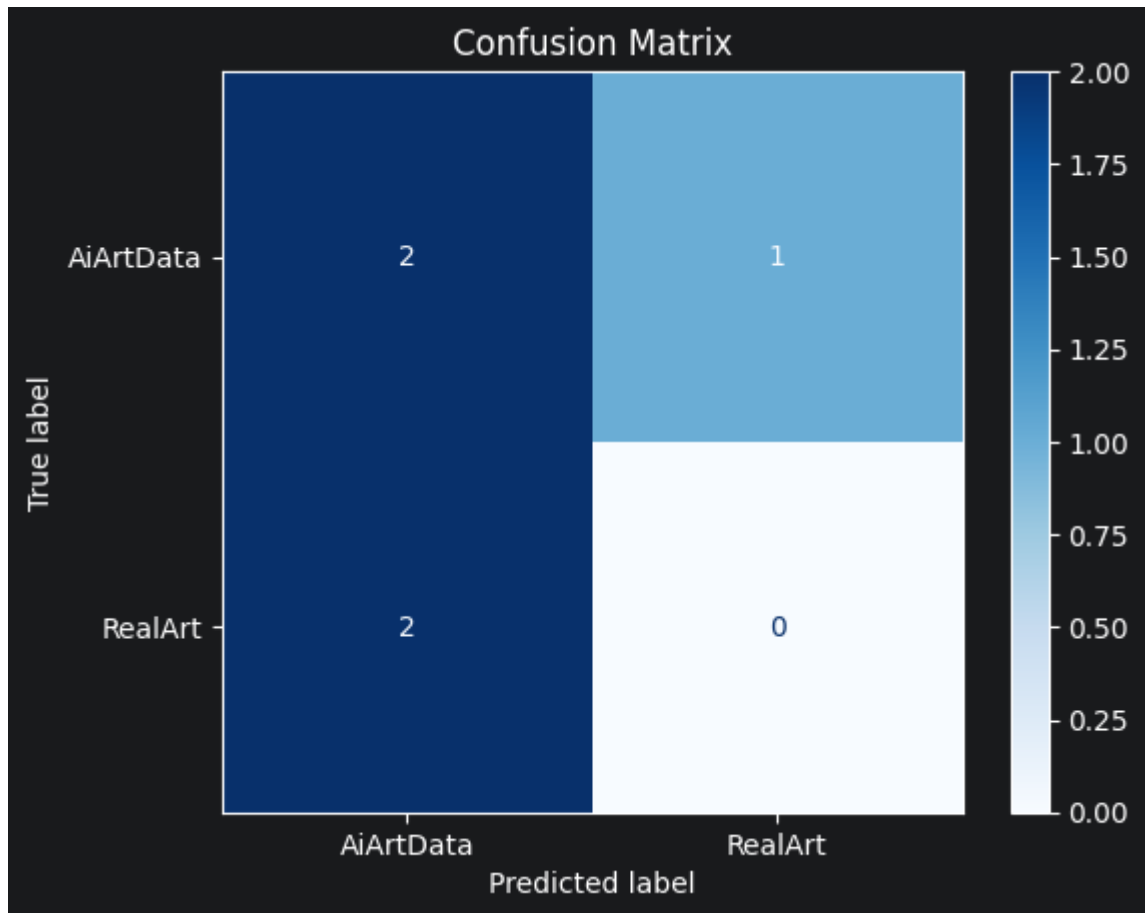
```
In [46]: fig, axes = plt.subplots(1, len(new_image_paths), figsize=(4 * len(new_image_paths), 4 * len(new_image_paths)))
if len(new_image_paths) == 1:
    axes = [axes]

for ax, image_path, (_, row) in zip(axes, new_image_paths, results.iterrows()):
    ax.imshow(load_img(image_path, target_size=IMG_SIZE))
    ax.axis('off')
    color = 'green' if row['correct'] else 'red'
    ax.set_title(
        f"True: {row['true_label']}\nPred: {row['predicted_label']}\nConf: {row['confidence']}"
        color=color, fontsize=9
    )

plt.suptitle('Model Predictions', fontsize=13)
plt.tight_layout()
plt.show()
```



```
In [47]: cm = confusion_matrix(true_indices, predicted_indices, labels=[0, 1])
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=[index_
disp.plot(cmap='Blues', values_format='d')
plt.title('Confusion Matrix')
plt.show()
```



Task 2b - Results on new images

After testing the model on our own images there are several patterns that are worth noting:

Accuracy: 2/5 (40%) - well below the 76% validation accuracy. The model correctly classified two of the three AI images and got both real photographs wrong.

Confidence is highest where it should be. The only confident, correct prediction (85.8%) is the stylized lion, an image with the saturated colors and clean illustrative style that dominate the AI training set. The framed AI artwork sits just barely on the right side of the 0.5 threshold at 54.5% confidence.

The "advanced generator" miss is informative. The AI-generated kitchen scene was misclassified, and crucially this was an image from a recent, high-quality generator that produces photorealistic output rather than illustrative output. When confronted with a photorealistic AI image, the model, mostly trained on illustrative AI art, has no learned cues to fall back on.

The model is biased toward predicting AI. Both real photographs were misclassified, and this is consistent with the bias we observed on the validation set (Real Art recall was

only 0.64).

Consequences and limitations

A 76% classifier misclassifies roughly 1 in 4 images, but our task 2b results suggest this number is optimistic for real-world use. On the five images we sourced ourselves, accuracy dropped to 40%, with both casual photographs misclassified as AI and a photorealistic AI image misclassified as real.

For applications with real-world stakes this model would be too unreliable to use as a definitive classifier. It should be treated as a screening or supporting tool that flags candidates for human review, and only on images stylistically similar to the training distribution.

Use of AI Tools

We used AI assistants (ChatGPT and Claude) primarily as a productivity tool for routine implementation work, so that we could focus our effort on the conceptually important parts of the assignment: model selection, evaluation strategy, and interpretation of results. Specifically, AI tools were used for:

- **Data cleaning implementation:** writing regex patterns for detecting gibberish, spam repetition, emoji spam, and non-English text.
- **Boilerplate EDA code:** formatting summary statistics, generating standard distribution plots, and writing print-statement scaffolding for inspecting flagged reviews and model scores.